



JAVA FUNDAMENTALS

1. What is Java?

Java is a **high-level, object-oriented programming language** used to build applications.

👉 Key idea:

Everything in Java is inside a class

2. What is a Class?

A **class is a blueprint** used to create objects.

```
public class HelloWorld {  
}
```

- class → keyword
- HelloWorld → class name (identifier)

📌 Rule:

- File name must match class name

👉 HelloWorld.java

3. What are Keywords?

Keywords are **reserved words** in Java.

👉 Examples:

class, public, static, void, if, else, for

✗ Cannot be used as identifiers

int class = 10; // ✗ ERROR

👉 Error:

error: <identifier> expected

4. What are Identifiers?

Identifiers are **names given to variables, methods, classes**

✓ Examples:

```
int age;  
String name;  
class Student {}
```

✗ Invalid:

```
int 1age; // ✗ starts with number  
int class; // ✗ keyword
```

👉 Error:

not a statement / identifier expected

5. RULES TABLE (CLASS, KEYWORDS, IDENTIFIERS)

Concept	Allowed	Not Allowed	Example (Valid)	Example (Invalid)	Error
Class Name	Letters, digits	Start with number	class Student	class 1Student	identifier expected

Keywords	Reserved words	Cannot rename	int age	int class	keyword error
Identifier	Letters, _, \$	No spaces/symbols	userName	user-name	illegal character



6. PRINT vs PRINTLN TABLE

Method	Behavior	Example	Output
print()	Same line	System.out.print("Hi")	Hi
println()	New line	System.out.println("Hi")	Hi (next line)
printf()	Formatted	System.out.printf("%d", 10)	10

Special Case

System.out.print(); // ❌ ERROR

System.out.println(); // ✅ valid

👉 Error:

no suitable method found for print()



7. LIST OF COMMON JAVA KEYWORDS

class, public, private, protected, static, void, int, double, float, boolean, if, else, switch, case, for, while, do, break, continue, return, new, this, super, try, catch, finally, throw, throws

⚠️ Reserved but unused:

const, goto

Literals

true, false, null

Literals are immutable in nature

8. MAIN METHOD (VERY IMPORTANT)

```
public static void main(String[] args)
```

Meaning:

- public → accessible everywhere
- static → no object needed
- void → returns nothing
- main → entry point
- String[] args → command-line arguments

Why Java introduced main()?

- 👉 JVM needs a **starting point**
- 👉 Execution always begins from main()

What if main() is missing?

```
class Test {}
```

✓ Compiles

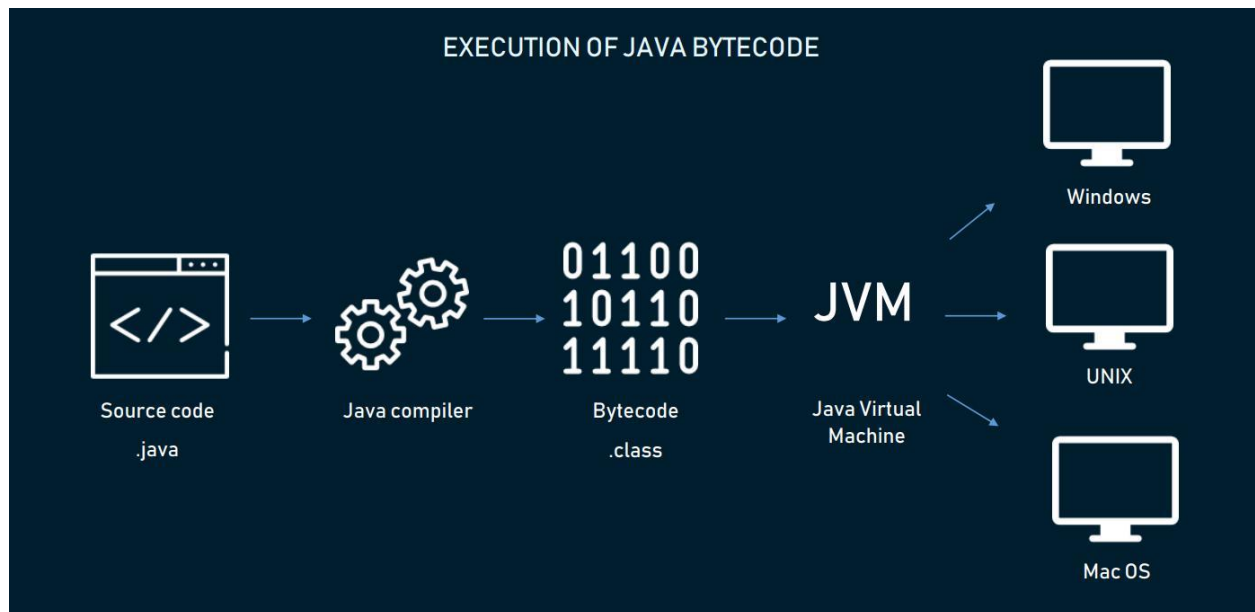
✗ Does NOT run

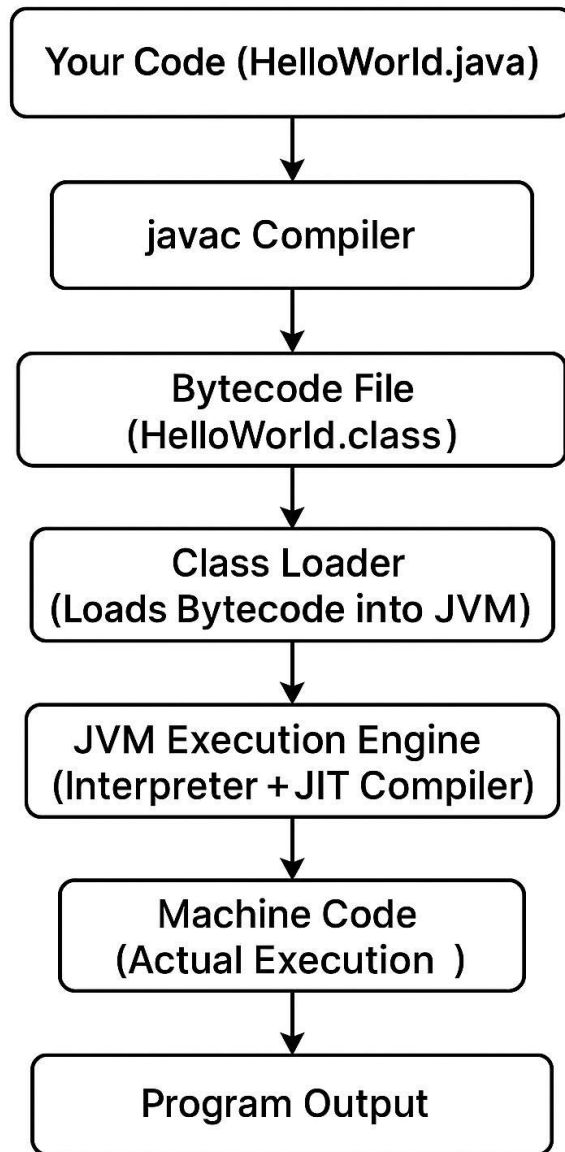
👉 Error:

Error: Main method not found in class Test



9. HOW JAVA EXECUTION WORKS





Step-by-step:

1. Write code → HelloWorld.java
2. Compile:

```
javac HelloWorld.java
```

3. Bytecode created → HelloWorld.class
4. JVM runs:

```
java HelloWorld
```

5. Output displayed



10. NUMBER OF CLASSES IN A FILE

✓ Allowed:

```
class A {}  
class B {}
```

✓ Compiles → creates:

```
A.class  
B.class
```

! Rule: Only ONE public class

```
public class A {}  
public class B {} // ✗ ERROR
```

👉 Error:

class B is public, should be declared in a file named B.java



11. SAME CLASS NAME?

✗ Not allowed:

```
class Test {}  
class Test {} // ✗ ERROR
```

👉 Error:

duplicate class: Test



12. NESTED CLASS (INNER CLASS)

```
class Outer {  
    class Inner {  
    }  
}
```

✓ Allowed

✓ Used for grouping logic

Types:

- Inner class
- Static nested class



13. WHAT-IF SCENARIOS (IMPORTANT)

Case 1: No main method

✓ Compile

✗ Run fails

Case 2: Multiple main methods

```
class A {  
    public static void main(String[] args) {}  
}  
class B {
```



```
    public static void main(String[] args) {}  
}
```

✓ Both can run:

java A

java B

Case 3: Wrong main signature

```
public static void main()
```

✗ Error:

Main method not found

Case 4: Class name ≠ File name

```
public class A {}
```

File: Test.java

✗ Error:

class A is public, should be declared in a file named A.java

Case 5: Using keyword as variable

```
int for = 10;
```

✗ Error:

not a statement

Case 6: Identifier with special char

```
int user-name;
```

✗ Error:

illegal character

Case 7: print() without argument

```
System.out.print();
```

✗ Compile error

Case 8: println() without argument

```
System.out.println();
```

✓ Prints blank line